

WEEKLY PROGRESS REPORT

Week 4: IPS - Data Purging, AI Hard Boundaries & Hybrid Stateful DPI

Lê Quý Tôn

April 26, 2026

Project: CyberSec Internship - Cygnus-Security
Related Issues: #23

1. Contribution Scope

This report details the architectural refinement of the Machine Learning pipeline and the implementation of a Hybrid Intrusion Prevention System (IPS). It addresses critical Machine Learning pitfalls such as Data Poisoning and Model Overfitting, and introduces a Stateful Behavioral Engine (Sliding Window) to mitigate the mathematical limitations of single Decision Trees.

2. Objectives of This Week

- Identify and eliminate Data Poisoning artifacts from the CIC-DNS-EXF-2021 PCAP extractions.
- Refactor the Feature Extraction vector to prevent “Shortcut Learning” by the AI model.
- Implement a Real-Time Sliding Window algorithm to evaluate query frequencies.
- Resolve network timing anomalies (Race Conditions and Timeout Traps) during live traffic simulations.

3. Detailed Work

3.1 Description

The AI model exhibited unpredictable behavior in live environments, occasionally flagging legitimate domains while ignoring weak Hex-encoded malware payloads. Deep analysis revealed that the training dataset was polluted with OS-level background noise. The system was overhauled into a two-tier Hybrid Engine: AI for static payload analysis (Lexical), and Python for behavioral frequency tracking (Stateful).

3.2 Design / Method

Data Pipeline Sanitization:

The training script was rewritten to filter out known OS noise (`ntp.org`, `.arpa`). Furthermore, the `Root_Length` feature was entirely removed. The AI is now restricted to a 3-dimensional vector: [`Subdomain_Length`, `Entropy`, `Digit_Ratio`]. This forces the AI to evaluate the actual obfuscated payload blindly.

Hybrid Stateful Architecture:

To overcome the “Hard Boundaries” of the ID3 algorithm, a Time-based Sliding Window was introduced. The system now requires 3 unique malicious payloads from the same IP within a 60-second window before classifying it as an active exfiltration attempt.

3.3 Implementation

- Updated `build_dataset_cic_v2.py` and retrained the model. Random Forest was explicitly utilized during the training phase solely as a statistical tool to evaluate Feature Importances, while the lightweight ID3 model was exported (`.pk1`) for high-speed production deployment.
- Configured `dnsmasq` to act as a simulated Authoritative Name Server, routing malicious domains perfectly to the Attacker VM.
- Upgraded the `traffic_simulator.sh` to utilize the `dig` command (`+time=1 +tries=1`) for rapid, timeout-resistant payload injection.

4. Results / Output

- The refined AI successfully ignored legitimate traffic and accurately identified Base64 DNS payloads.
- The Sliding Window accurately tracked attacker IPs (Hit 1/3, 2/3), absorbing temporary anomalies and severing the connection via `iptables` upon reaching 3/3, terminating the exfiltration after less than 150 bytes of data transfer.

5. Analysis

The week’s findings highlighted the operational difference between Theoretical AI and Applied Security. Decision Trees (ID3) possess strict mathematical thresholds that can be bypassed by attackers altering their encoding schemes. However, by layering a Behavioral Frequency Counter over the AI, the system compensates for individual packet misclassifications. Attackers are forced to send thousands of packets to extract a file; capturing just 3 packets out of a burst is sufficient for the Stateful Engine to trigger a permanent network block.

6. Issues & Fixes

- **Issue:** Data Poisoning and Overfitting (AI falsely flagged clean domains due to long root domains).

- **Fix:** Removed `Root.Length` feature and purged OS noise from the CIC training set.
- **Issue:** Sliding Window Race Condition & A/AAAA Duplicates.
- **Fix:** Upgraded the counter to track “Unique Domains” via Python Sets, expanded the Time Window to 60s, and switched the traffic simulator to `dig`.
- **Issue:** Console log spam after an IP was blocked.
- **Fix:** Implemented an IP-Whitelist bypass inside the Python listener script to silently ignore packets from already penalized IPs.

7. Reusable Content ★

(Handling Sliding Window Race Conditions in IPS)

When implementing frequency-based detection (Sliding Windows), standard packet counting often leads to False Positives. A single DNS resolution request typically generates simultaneous IPv4 (A) and IPv6 (AAAA) queries, artificially doubling the hit counter. Furthermore, network timeouts can cause the window to expire before the threshold is met (Timeout Trap).

To build a robust Stateful DPI Engine:

1. **Track Unique Signatures:** Instead of counting raw packets, group and count unique payload signatures (e.g., Unique Subdomains).
2. **Account for Network Latency:** Extend the time window slightly beyond the standard timeout threshold of the simulated protocols (e.g., 60 seconds for DNS).

8. Next Step (Future Work)

- Design and develop a Graphical User Interface (GUI) using **Streamlit** to visualize real-time DNS traffic, entropy graphs, and blocked IP tables for the final project demonstration.
- Explore the feasibility of replacing the ID3 model with **Random Forest** in the production pipeline to dynamically soften hard classification boundaries, evaluating its impact on Gateway latency.